Report on CUDA Performance

> Note: S&E is closed now so I'm using SCALE as a compatibility layer for running CUDA on amd gpus. It does have very similar performance compared to raw CUDA (+-5% difference) so hopefully it's fine. Results will probably vary a bit when running raw CUDA, but the overhead is *very* minimal.
>
> Another Note: I thought S&E was closed but I forgot I can still scan in and ran in a few minutes before it closed and *also* and got results on the Nvidia GPUs. I wrote the first sections of this report based on the SCALE/AMD results. I did however, compare both results from SCALE and CUDA which is interesting. Hope that's okay.

Results

SCALE/AMD

Matrix Width	Block Size	CPU Time (ms)	GPU Total Time (ms)	Total Data Transfer Time (ms)	GPU Processing Time (ms)	Speedup
4	2	0.000000	0.743795	0.653344	0.090451	0.000000
4	4	0.000000	0.717106	0.645991	0.071115	0.000000
8	2	0.000000	0.605844	0.534399	0.071445	0.000000
8	4	0.000000	0.696035	0.625221	0.070814	0.000000
8	8	0.000000	0.712006	0.632325	0.079681	0.000000
16	2	0.001000	0.691535	0.615422	0.076113	0.001446
16	4	0.001000	0.724218	0.650579	0.073639	0.001381
16	8	0.001000	0.704621	0.628317	0.076304	0.001419
16	16	0.001000	0.719280	0.637275	0.082005	0.001390
32	2	0.007000	0.734638	0.656180	0.078458	0.009529
32	4	0.007000	0.657281	0.578392	0.078889	0.010650
32	8	0.011000	0.718779	0.638307	0.080472	0.015304

Matrix Width	Block Size	CPU Time (ms)	GPU Total Time (ms)	Total Data Transfer Time (ms)	GPU Processing Time (ms)	Speedup
32	16	0.008000	0.720842	0.644187	0.076655	0.011098
32	32	0.007000	0.730370	0.648315	0.082055	0.009584
64	2	0.067000	1.195338	1.098235	0.097103	0.056051
64	4	0.063000	0.832303	0.755027	0.077276	0.075694
64	8	0.063000	0.861127	0.771678	0.089449	0.073160
64	16	0.062000	0.841280	0.756640	0.084640	0.073697
64	32	0.062000	1.281141	1.188366	0.092775	0.048394
128	2	0.941000	1.310146	1.161996	0.148150	0.718241
128	4	0.948000	1.276472	1.179509	0.096963	0.742672
128	8	0.951000	1.266602	1.160512	0.106090	0.750828
128	16	0.964000	1.283926	1.184268	0.099658	0.750822
128	32	0.948000	1.273608	1.166415	0.107193	0.744342
256	2	10.386000	3.574534	2.940035	0.634499	2.905554
256	4	10.682000	3.197030	2.946667	0.250363	3.341226
256	8	10.412000	3.083035	2.913314	0.169721	3.377192
256	16	10.228000	3.067516	2.893718	0.173798	3.334294
256	32	10.351000	3.079268	2.904528	0.174740	3.361513
512	2	214.921005	5.985271	1.419933	4.565337	35.908318
512	4	215.509995	2.608099	1.372364	1.235735	82.631065
512	8	213.274994	2.039825	1.366142	0.673683	104.555534
512	16	213.125000	2.079951	1.424923	0.655028	102.46636

Matrix Width	Block Size	CPU Time (ms)	GPU Total Time (ms)	Total Data Transfer Time (ms)	GPU Processing Time (ms)	Speedup
						2
512	32	214.619003	2.081383	1.445752	0.635631	103.113655
1024	2	3922.832031	38.092010	2.255702	35.836308	102.983063
1024	4	4308.101074	11.823953	2.267514	9.556438	364.353729
1024	8	4340.241211	7.191860	2.378604	4.813256	603.493530
1024	16	4046.583008	7.150482	2.135546	5.014936	565.917542
1024	32	4368.100098	7.328420	2.202403	5.126017	596.049377
2048	2	39662.906250	391.232574	8.024073	383.208496	101.379356
2048	4	39876.015625	105.535301	8.106600	97.428703	377.845276
2048	8	39983.656250	57.436951	8.265071	49.171883	696.131226
2048	16	39673.023438	57.849400	8.109948	49.739452	685.798340
2048	32	39718.160156	57.886402	7.978932	49.907467	686.139709

CUDA/NVIDIA

Matrix Width	Block Size	CPU Time (ms)	GPU Total Time (ms)	Total Data Transfer Time (ms)	GPU Processing Time (ms)	Speedup
4	2	0.001000	868.237793	0.577472	867.660339	0.000001

Matrix Width	Block Size	CPU Time (ms)	GPU Total Time (ms)	Total Data Transfer Time (ms)	GPU Processing Time (ms)	Speedup
4	4	0.000000	0.238656	0.119456	0.119200	0.000000
8	2	0.001000	0.230272	0.112448	0.117824	0.004343
8	4	0.001000	0.259584	0.135456	0.124128	0.003852
8	8	0.001000	0.231648	0.114560	0.117088	0.004317
16	2	0.009000	0.230752	0.107680	0.123072	0.039003
16	4	0.009000	0.224928	0.103072	0.121856	0.040013
16	8	0.009000	0.237888	0.111904	0.125984	0.037833
16	16	0.009000	0.234368	0.106048	0.128320	0.038401
32	2	0.068000	0.241760	0.113856	0.127904	0.281271
32	4	0.068000	0.237920	0.112480	0.125440	0.285810
32	8	0.068000	0.261024	0.128928	0.132096	0.260512
32	16	0.068000	0.229184	0.108640	0.120544	0.296705
32	32	0.068000	0.273056	0.147840	0.125216	0.249033
64	2	0.546000	0.309280	0.172096	0.137184	1.765391
64	4	0.556000	0.312128	0.171040	0.141088	1.781320
64	8	0.554000	0.244128	0.121952	0.122176	2.269302
64	16	0.547000	0.256704	0.127200	0.129504	2.130859
64	32	0.553000	0.248768	0.122336	0.126432	2.222955
128	2	5.079000	0.336672	0.188032	0.148640	15.085899
128	4	5.072000	0.356480	0.210752	0.145728	14.228007
128	8	5.673000	0.521024	0.345120	0.175904	10.888174

Matrix Width	Block Size	CPU Time (ms)	GPU Total Time (ms)	Total Data Transfer Time (ms)	GPU Processing Time (ms)	Speedup
128	16	5.068000	0.387040	0.237440	0.149600	13.094253
128	32	5.052000	0.310272	0.186528	0.123744	16.282490
256	2	52.935001	0.688320	0.396640	0.291680	76.904640
256	4	53.692001	0.599392	0.408384	0.191008	89.577438
256	8	52.514999	0.569760	0.400576	0.169184	92.170395
256	16	54.625000	0.591072	0.424800	0.166272	92.416832
256	32	51.883999	0.666432	0.501600	0.164832	77.853409
512	2	499.231995	3.061376	1.578944	1.482432	163.074387
512	4	514.460999	2.146816	1.512736	0.634080	239.639069
512	8	517.989990	1.692992	1.352288	0.340704	305.961273
512	16	500.317993	1.645408	1.342816	0.302592	304.069244
512	32	516.382996	1.681056	1.365632	0.315424	307.177734
1024	2	5223.592773	15.153408	3.932352	11.221056	344.714050
1024	4	5463.710938	7.371808	3.958048	3.413760	741.162964
1024	8	4858.433105	5.960064	3.959136	2.000928	815.164612
1024	16	5045.541016	5.403072	3.904928	1.498144	933.828125
1024	32	5491.776855	5.477056	3.980032	1.497024	1002.687744
2048	2	104186.328125	99.369286	14.181280	85.188004	1048.476196
2048	4	111640.070312	40.604156	14.229888	26.374271	2749.473877
2048	8	106015.5390	28.041344	14.865696	13.175648	3780.68676

Matrix Width	Block Size	CPU Time (ms)	GPU Total Time (ms)	Total Data Transfer Time (ms)	GPU Processing Time (ms)	Speedup
		62				8
2048	16	103000.031250	25.579103	14.196064	11.383040	4026.725586
2048	32	104565.484375	25.970592	14.368672	11.601920	4026.303467

Graphs

> Charts are generated with Chart.js. If you want that code, let me know

Fig. 1 (SCALE/AMD): CPU vs GPU time

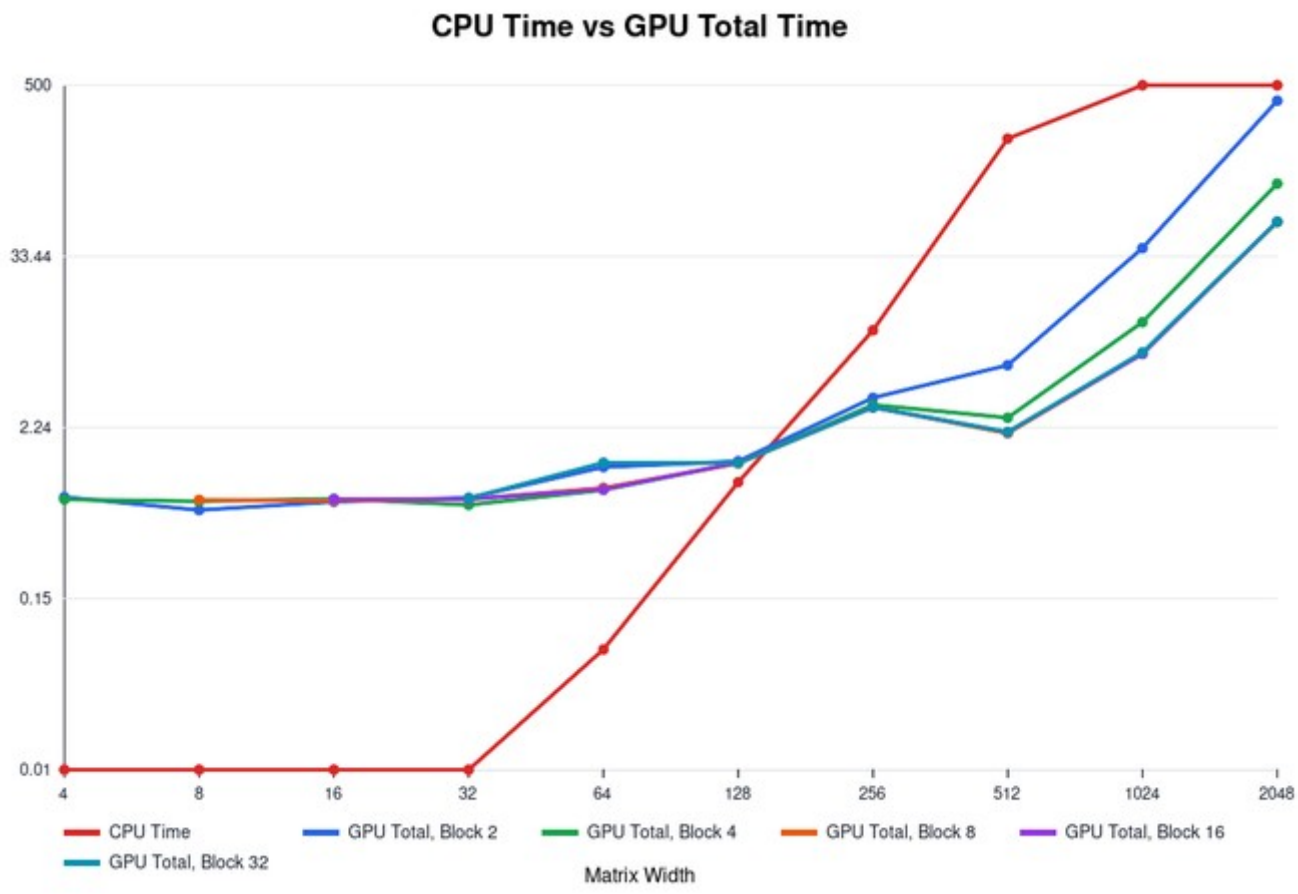


Fig. 2 (SCALE/AMD): Block size vs GPU time

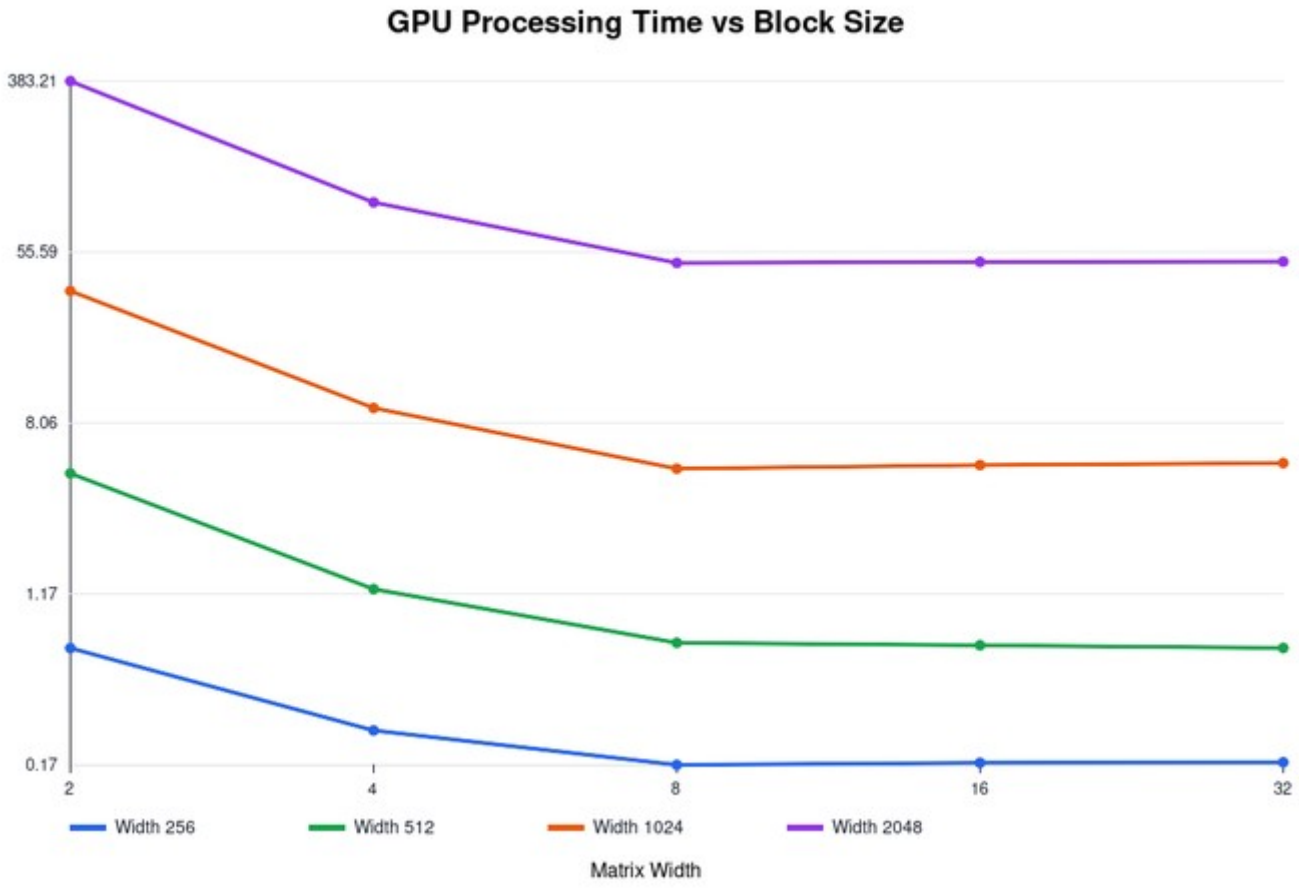


Fig. 3 (SCALE/AMD): Transfer vs GPU Time

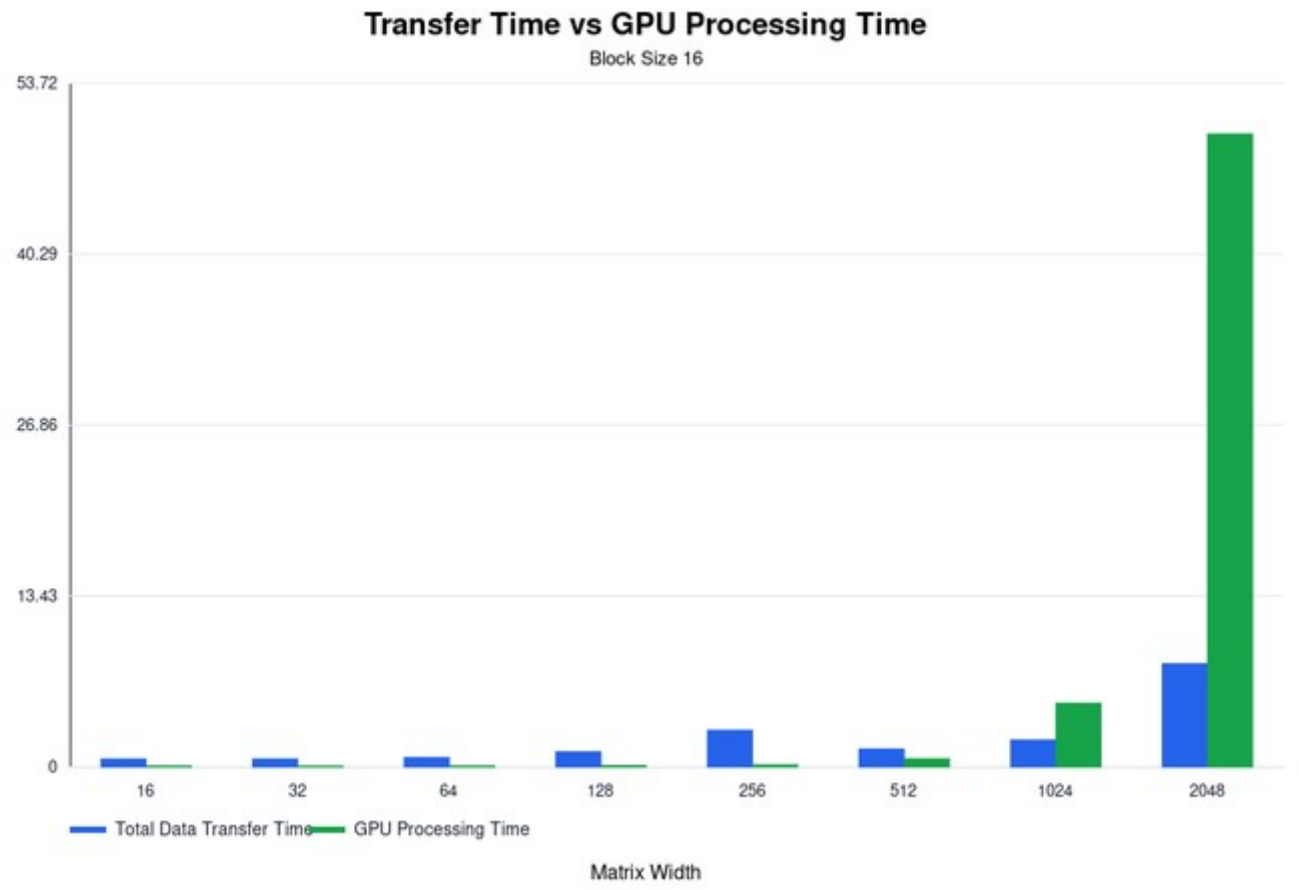


Fig. 4 (CUDA): CPU vs GPU time

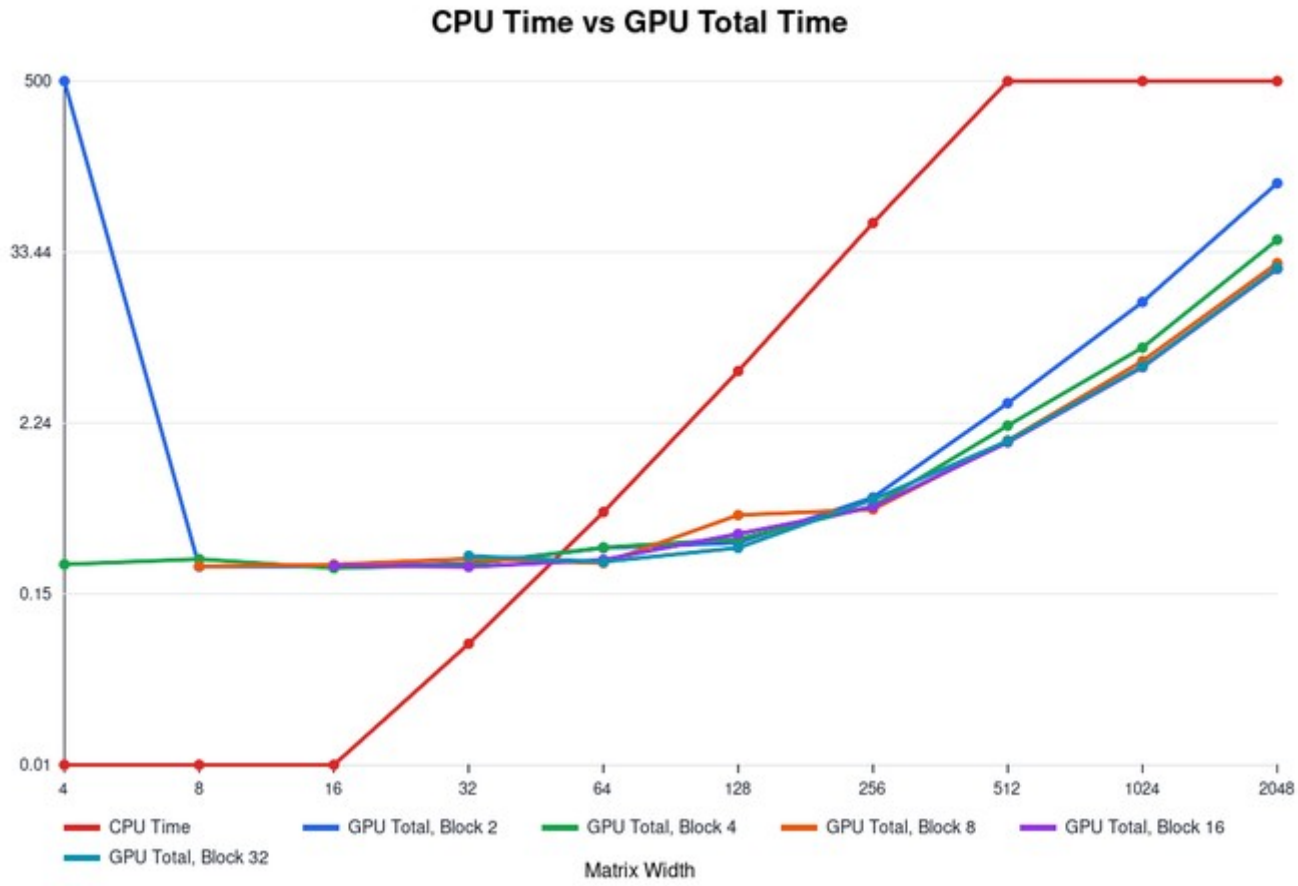


Fig. 5 (CUDA): Block size vs GPU time

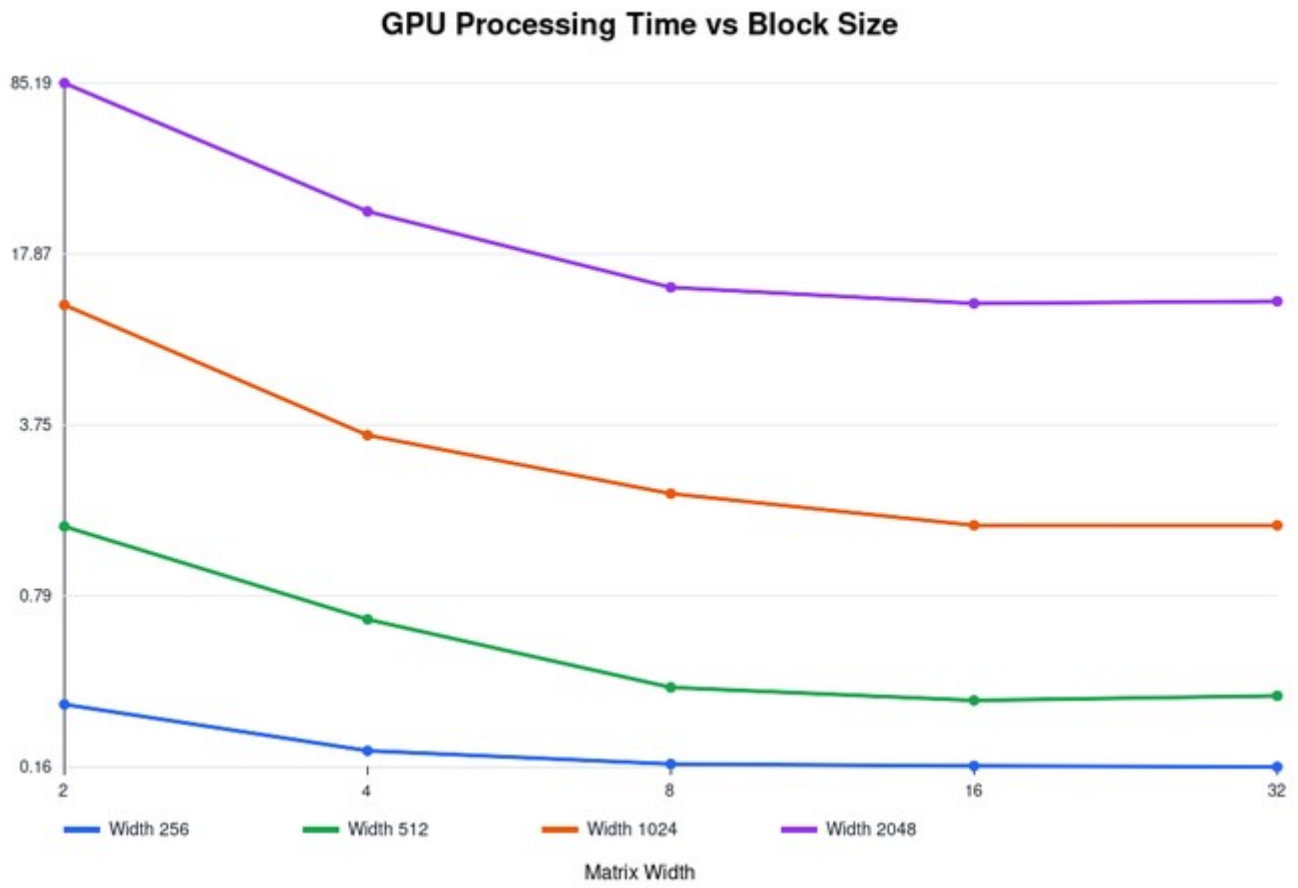
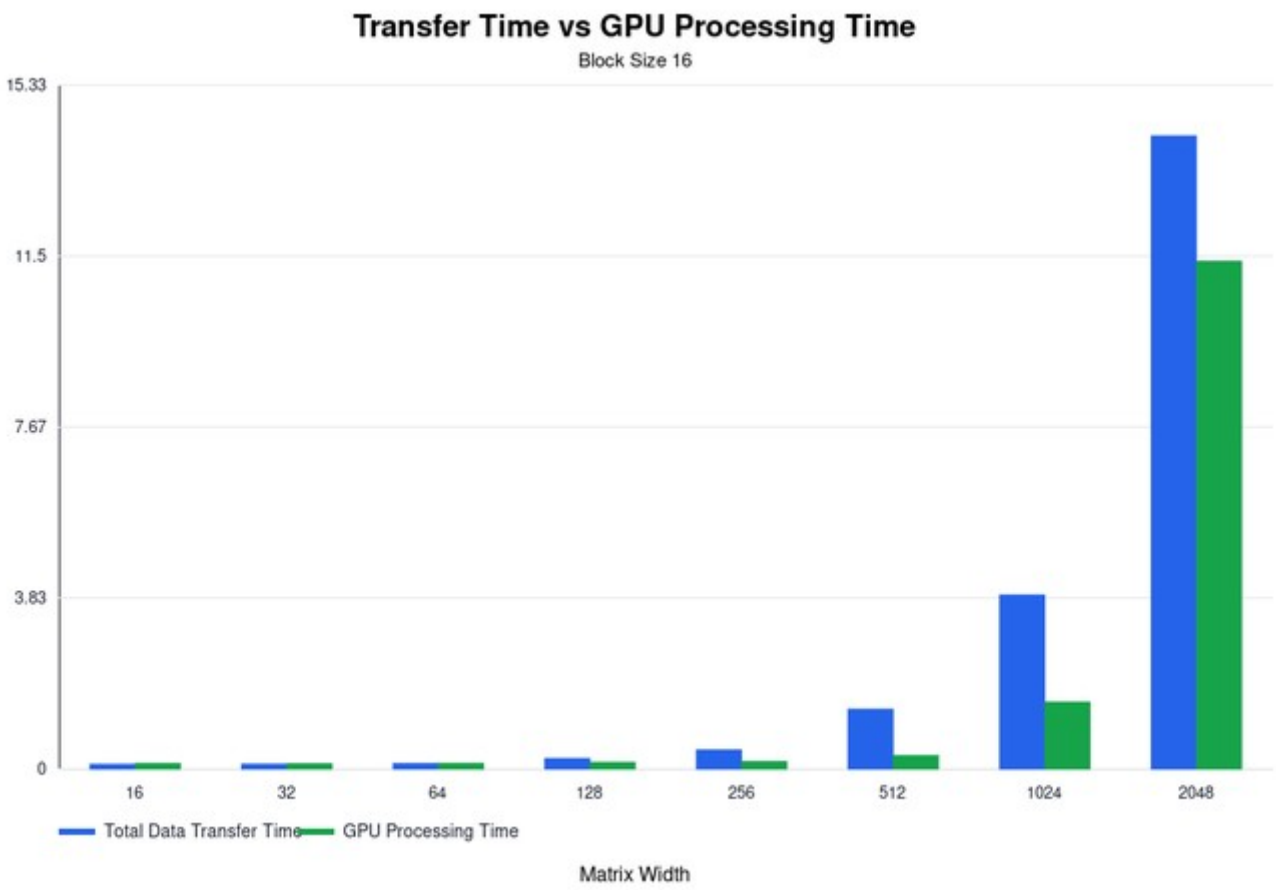


Fig. 6 (CUDA): Transfer vs GPU Time



Analysis

First, it should be noted that for *Fig. 1*, CPU Time hits both an upper and lower limit, so when it shows `0.01`, it actually means sub-0.01 and when 500, it means plus-500.

When multiplying smaller (<128 width) matrices, the CPU time clearly beats the GPU time (Fig. 1). The primary reasons for this are:

1. GPUs are optimized for extreme parallelism. A small matrix cannot be split up into worthwhile chunks across many threads, so the GPU regresses performance.

2. Crossing the memory bounds between Host and Device memory likely takes more time than the entire computation itself, so most of the results are initializing the kernel, transferring data, multiplying, and transferring back.

In the professional world, I run into this problem a lot. Typically, I write lots of high-performance WebAssembly libraries and runtimes.

The main factor in deciding whether some code is worth running in WebAssembly is exactly the same: if the transfer time exceeds or equals the computation time, then it's likely not beneficial. The same applies to GPUs, which is why we are seeing these results.

However, when providing the GPU with enough workload so that transfer time < computation time, the handles it perfectly.

Taking another look at Fig. 1, the CPU and GPU time intersects at a x-coordinate of around 150. That means that somewhere between a matrix width of 128 and 256, the transfer overhead is superceded by the gpu time (or rather ability to chunk and parallelize data).

Another reason is WARPs. Even though I'm running a small compatibility layer, SCALE still creates a WARP-equivalent of 32-threads (I looked it up and it's called a wavefront and on my system, it has 32 threads https://youtu.be/uu-3aEyesWQ?t=184). The problem is that both a 2x2 and 4x4 block size cannot fully fill a WARP, which means that they do not fully take advantage of the hardware.

Say if I ran this matrix multiplication algorithm with a 4x4 block size, it would only fill half a warp. Now, if I ran it in 8x8 blocks, it would take 64 threads which takes full advantage of two WARPs. If we take a look at the actual data, we find that for 6/10 of the time (with 2 of those 10 being invalid aka negative speedup, so in reality, 6/8), an 8x8 block size outperforms all other runs.

This though, begs the question: if say, I use a block size of 16x16 (8 WARPs), and matrix width of 2048, would it not be faster since it utilizes 8 WARPs simulaneously?

In Fig. 1, we see that after a 8x8 block configuration, results sort of plateu, but if we take a look at Fig. 3, we see that the data transfer time nearly triples between 1024 and 2048.

In all honesty though, I'm not too certain. My best guess is that the tail section of each computation only uses about half the allocated WARPs which degrades performance. (Do you have any insights into this, professor?)

If we take a look at complexity, both the CPU and GPU implementations do the same amount of work, so they run at O(n^3) complexity.

The difference is that the GPU is able to distribute that load across thousands of threads. (My system has around 200k threads, so it has plenty of room), so it can effectively spread that complexity and run at a constant rate (which is why we see results plateu at the end).

For the GPU, space complexity is also a problem, which seems to increase at rate of something like O(n^2) or so. Again, the GPU is able to compensate for this because it also spreads that O(n^2) complexity across the same threads. Pretty cool.

That said, if a large enough matrix isn't provided, both the time and space complexity can't be properly distributed which hurts performance a lot.

Differences between AMD and CUDA results

When comparing SCALE/AMD and CUDA results, I found some noticable differences.

Most obviously, the CPU baselines are very different. For example, we get the following results (Fig 1&5):

Average CPU @ 256x256:

MINE: ~10.4ms

LAB: ~53ms

The CPU in the lab computers is around 5.1x slower.

Or, take this example:

Average CPU @ 2048x2048

MINE: ~39s

CUDA: ~106s

So, ~2.7x slower

The reason that CUDA shows such massive speedups is one because it's running natively, but also that the CPU is much worse.

Secondly, GPU overtakes the CPU at around at around a 64x64 matrix. HOWEVER, the CPU in the lab is very different from mine. If, for example, my cpu was the baseline, it would only become *worse* than the GPU at around a 128x128 matrix.

When looking at the charts, I also saw a noticable outlier (width: 4, block: 2) at 868ms which I suspect is a side effect of the GPU switching from lower wattage to higher or a warmup/optimizing stage.

Probably the best statistic here is comparing CUDA and SCALE kernel times since they are unaffected by the CPU. Granted, SCALE is a runtime, so the GPU code is not running natively, but we still get some interesting results (comparing best results):

512x512: SCALE: 1.26ms CUDA: 0.31ms

1024x1024: SCALE: 7.15ms CUDA: 5.40

2048x2048: SCALE: 57.43ms CUDA: 25.57ms

In SCALE, the best block size was typically 8x8 and if not that, 16x16. However, in CUDA, that changes and shifts towards 16x16 and if it's a larger matrix, 32x32.

If I had a different CUDA GPU to test with, I'm sure we'd see similar results where different GPU generations prefer different block sizes. Not sure if they do it in the wild, but it'd probably be good practice to run GPU code against multiple platforms and choose a good middle ground for block sizes.

Say if you hard code a block size value that prefers blackwell architecture, but your user runs pascal, there might be some big problems.

Oh, also, small block sizes are usually always bad since they can't fill a single WARP. Typically, filling out a warp (aka making sure block size uses multiples of 32 threads) really does help performance.

Lastly, transfer times in CUDA appear to be a good bit faster for small payloads, but for large payloads, SCALE actually wins even though it's running on a compatibility layer. I suppose transferring between host and device is always costly no matter the hardware, so using shared memory would probably be a big benefit here.